

REVIEW

Independent Code Review — Committer's 3 stable landed PARTs

Reviewer role: §11.4.142 / §11.4.125 independent code-review (author = big committer; reviewer = separate, READ-ONLY, non-committing per §11.4.84). **Run:** 2026-06-16T16:07:54Z. **Main checkout HEAD at review time:** 4fbe581 (in-flight committer's bug-class work — OUT of scope). **Scope reviewed:** af2ef7f (PART A), 04f71e9 (PART B), c2aa7c8 (PART C). nezha-arc iter1-4 + in-flight 4fbe581 bug-class work were NOT reviewed (per instruction). **Method:** git show each SHA → read changed source + tests → build/test the touched packages → §1.1 mutation-prove each new test in a throwaway worktree → exercise the credential hook with RED/GREEN inputs → docs_chain verify features + headline-math audit. No main-checkout tracked file was edited/staged/committed by the reviewer (mutations isolated to /tmp worktree, removed; zero residue confirmed).

Verdicts

PART	SHA	Verdict
A — §11.4.153 video waves 4-6 (43→54)	af2ef7f	GO with FINDING (non-blocking, doc-integrity)
B — secret-scrub + credential hook	04f71e9	GO
C — BUG-MULTIPASS-DEFAULT-MODEL + BUG-FB2-HARDCODED-LANG	c2aa7c8	GO (one owed item: nezha runtime re-validation, honestly deferred)

No PASS-bluff, no FAIL-bluff, no bluff-gate found. Two genuine product bugs are correctly root-caused, fixed robustly, and covered by tests that provably catch the negation (§1.1 mutation-confirmed). The secret scrub is complete and the new credential hook genuinely blocks recurrence.

PART C — c2aa7c8 — GO (deepest scrutiny)

BUG-MULTIPASS-DEFAULT-MODEL (cmd/unified-translator/main.go)

- **Correctness:** root cause is FACT-accurate — the polish path built its LLM client with `config.Model` (global `-model` default `gpt-4`), which `NewLLMTranslatorWithConfig`'s `ValidModels` whitelist rejects for any non-openai provider, so the polish silently no-opped while the step showed green (§11.4/§11.4.138 PASS-bluff). `resolvePolisherModel(provider, requested)` substitutes the provider's canonical first valid model ONLY when the requested model is invalid for that provider; keeps an explicit-valid model; passes through providers with no whitelist. `llm.ValidModels` / `llm.Provider` are exported and used identically by the per-provider clients — the substitution mirrors exactly what the main path gets from the bridge. No solve-A-create-B: a *genuine* polish failure now marks the step `Success=false` (✗) instead of (✓) while still preserving the base translation. `addStep` appends a pointer-stable `*TranslationStep` (`Success` defaults `false`), so the failed step still appears in the report. (Micro-nit, non-blocking: the failure branch inlines what the existing `stepError` helper already does — could call `stepError(mpStep, ...)` for consistency; functionally identical.)
- **Test genuineness (§1.1 mutation-PROVEN):**
 - `TestRunMultiPass_DefaultModelStillPolishes` drives a real `httpTest` OpenAI-compatible server, `provider=deepseek`, `model=gpt-4` (the default), and asserts the LLM was actually hit (`hits != 0`) AND the polished text was applied AND `output ≠ input`. **Mutation:** neutered `resolvePolisherModel` to `passthrough` → test FAILED with the exact prefix error `model 'gpt-4' is not valid for provider 'deepseek'`. Not a bluff.
 - `TestResolvePolisherModel` covers the full case-space (valid kept / invalid→default / empty→default / `gemini` / `openai-gpt4-kept` / unknown-provider `passthrough`). All 6 sub-cases RUN (verbose-confirmed, not skipped).

BUG-FB2-HARDCODED-LANG (pkg/api/handler.go translateFB2)

- **Correctness:** root cause FACT-accurate — handler called `createTranslator(provider, model, "", "")` + hardcoded `sourceLang=ru/targetLang=sr` for the prep path + `book.Language="sr"`, so every FB2 request produced Serbian. Fix parses `source_lang/target_lang`, **validates via `language.ParseLanguage` BEFORE building the translator** (unknown code → 400, no translator built — never a silent fallback), threads the RESOLVED codes (defaulting to `ru/sr` only when BOTH omitted) through `createTranslator`, the preparation-aware translator, and `book.Language`. `createTranslator`'s own `ru→sr` default only fires on both-empty and is now superseded by the handler always passing resolved non-empty codes — consistent, no double-default conflict. `language.Russian/Serbian/ParseLanguage` all exist. (Micro-nit, non-blocking: the `prepConfig` struct literal still sets `SourceLanguage:"auto"/TargetLanguage:"en"` then immediately overwrites both — dead initializer, harmless.)

- **Test genuineness (§1.1 mutation-PROVEN):**

TestTranslateFB2_HonorsRequestedLangs drives the real handler via gin + real multipart, recording the actual task.SourceLang/task.TargetLang the bridge factory was asked to build, across 5 cases (no-langs→ru/sr; en→es; en→de; target-only-fr keeps default ru source; klingon→400 + ZERO translator built). **Mutation:** reverted the call site to createTranslator(provider, model, "", "") → test FAILED with source threaded as "ru", want "en" (en→es, en→de) and target threaded as "sr", want "fr" (target-only). Catches the negation precisely. All 5 sub-cases RUN.

Owed (honestly deferred, not a defect of this commit)

- The FB2 fix is a server path; the commit message explicitly states "nezha rebuild+reboot+sink-side re-validation **follows in this same stream.**" At c2aa7c8 the live es→Spanish / de→German / klingon→400 nezha runtime evidence (docs/qa/) does **not yet exist** — it is the §11.4.108 RUNTIME-on-clean-target layer, honestly owed to the in-flight stream. The unit-level contract test proves the SOURCE+ARTIFACT layers. **Committer follow-up:** land the live nezha FB2 re-validation evidence before closing the FB2 row / tagging.

Build/test

- go build ./pkg/api/ ./pkg/translator/... ./pkg/verification/ ./cmd/unified-translator/ → clean.
- go test ./pkg/api/ ./pkg/translator/llm/ ./pkg/verification/ ./cmd/unified-translator/ -count=1 → all 4 GREEN.
- (NOTE — not a PART-C/B/A defect: go build ./... over the WHOLE tree fails with pkg/ebook: undefined: stripLeadingTitle — that is the in-flight committer's UNCOMMITTED working-tree edit to pkg/ebook/{epub,html}_parser.go, OUT of review scope. None of the 3 reviewed commits touch pkg/ebook.)

PART B — 04f71e9 — GO

Secret scrub (§11.4.10 / §11.4.30)

- git grep <leaked-token-redacted-§11.4.10> (live token) over all TRACKED files → **EMPTY** (value never printed in this report either).
- All 6 secret-bearing binaries (build/ebook-translator, ebook-translator, download-model, test-model, test-enhanced-conversion, test-simple-translation) confirmed **untracked** (git ls-files --error-unmatch fails for each).
- .gitignore now ignores /build/ + root-anchors the 6 binary names (GAP2 root cause fixed — was per-file only).
- Awareness docs (docs/qa/secret_scrub_*/PLAN.md, docs/scripts/credential_scan.md) carry only the MASKED form WhiteSnake<REDACTED-§11.4.10> — never the live 8587.

Credential hook genuinely prevents recurrence (§11.4.10.A clause 5, §11.4.75 L1/L3)

- Read scripts/git_hooks/credential_scan.sh — closed pattern set: leaked-class WhiteSnake[0-9]{2,}, inline --password flag (digit/len≥8 guard, word-boundary so prose "SSH-password" doesn't match), *_PASS/*_PASSWORD/Password assignment, API-key shapes (sk-,AKIA,ghp_,xox[baprs]-). is_env_or_placeholder correctly exempts \${VAR}/\${VAR}/empty/<redacted...>/YOUR_/CHANGEME.
 - **Anti-bluff RED/GREEN proven by reviewer (independent of committer's claim):**
 - RED (all → exit 1, value masked): --password <leaked-token-redacted-§11.4.10> (leaked-class), --password <example-pw-redacted> (inline literal), REMOTE_PASS="<example-redacted>" (assignment), sk-... (API-key shape).
 - GREEN (all → exit 0, no output): --password "\$ {SSH_WORKER_PASSWORD}", <redacted-per-11.4.10>, PASSWORD="".
 - pre-commit (staged-diff scan) + pre-push (per-range changed-file scan, catches --no-verify bypass) + idempotent install_git_hooks.sh all present; all 4 scripts pass bash -n (§11.4.67). The hook is genuinely effective.
 - **Minor blast-radius note (non-blocking follow-up):** IGNORE_RE self-exempts the *entire* scripts/git_hooks/ dir and any docs/qa/secret_scrub_* path. Justified (hook-internal + masked-awareness paths) but means a future real secret committed INTO those paths would go unscanned. Consider narrowing the exemption to the specific doc files rather than whole directories.
-

PART A — af2ef7f — GO with FINDING (doc-integrity, non-blocking)

What is correct

- Headline math is sound: Status.md Anti-bluff note $43 - 1 + 3 + 4 + 5 = 54$; Status_Summary $43 - 1 + 12 = 54$ ($3+4+5=12$). **No over/under-count in the headline.** Status_Summary.md headline is internally consistent at 54 (lines 17, 34, 61).
- docs_chain verify features → **features in-sync** (built the docs_chain CLI from its submodule, ran against current HEAD). The 8-node chain (Status.md/html/pdf/docx + Status_Summary.{md,html,pdf,docx}) is export-synchronized; DOCX is present per §11.4.153.
- New PASS rows (multi-language-pair, fb2, convert/script) cite real .mp4 artefacts at /Volumes/T7/Downloads/Recordings/ with ffprobe/frame detail per §11.4.107; the §11.4.138 multipass demotion (-1) correctly matches PART-C's confirmed bug.

FINDING (intra-document numeric drift — §11.4.6 precision / §11.4.60 / §11.4.91)

The SAME document carries THREE different "video-confirmed" counts that must all agree but do not. Confirmed present in the committed af2ef7f blob (not the in-flight edit):

- **Status.md line 8 (Anti-bluff headline): 54**  authoritative.
- **Status.md line 30 (Coverage-summary | Video-confirmed | cell): 43**  stale — never bumped this commit; its own prose math even terminates at 37/41/43 (Rev-14).
- **Status.md line 898 (caveat narrative): "Video coverage is 30 feature rows" / "30 features are confirmed"**  very stale (Wave-2 era).
- **Status_Summary.md carries the same drift:** line 128 "Video-confirmation 43/496", line 140 "43 / 496 = 8.7%" — contradicting its own 54 headline.

This is NOT a feature-correctness PASS-bluff (the 54 rows + their evidence are genuine; the headline is right). It IS a real doc-integrity defect: a reader pulling the Coverage-summary cell gets 43, the caveat gets 30, the headline gets 54 — precisely the summary-must-agree-with-headline drift §11.4.91/§11.4.60 forbids. docs_chain verify cannot catch it (it only checks md→html/pdf/docx export sync, not intra-document numeric consistency), so "in-sync" passed while the doc self-contradicts.

Committer follow-up (non-blocking for these landed commits, but must reconcile before the release-gate sweep): update the Status.md Coverage-summary Video-confirmed cell (43→54) + the line-898 caveat (30→54), and the Status_Summary lines 128/140 (43→54), then re-run docs_chain. Recommend a small intra-doc-consistency check (grep that all "video-confirmed N" tokens in a Status doc agree) since the export-sync gate structurally cannot.

§11.4.84 quiescence statement

All mutation experiments ran in a detached /tmp git worktree (replace-dirs repointed to the main checkout, removed with `git worktree remove --force`). Post-run grep of the main checkout's `pkg/api/handler.go + cmd/unified-translator/main.go` for mutation markers (`// MUT, MUTATED` for paired, `_ = "unreachable"`) → EMPTY. The reviewer edited/staged/committed NO main-checkout tracked file. This REVIEW.md is the only file written, and is not committed by the reviewer.